Arrays, Slices et Maps

---

1. Les Tableaux (`Arrays`) : Rigides et copiés par valeur

En Go, un tableau (`[N]T`) est une séquence d'éléments de même type dont la **longueur N fait partie intégrante du type** et doit être connue à la compilation.

1.1 Différence fondamentale avec le langage C

- **En C :** Une variable tableau (`int tab[5]`) est implicitement convertie en un pointeur vers son premier élément (`&tab[0]`) lorsqu'on la passe à une fonction. Aucune copie des éléments n'est effectuée.
- **En Go :** Un tableau est une **valeur brute**. Le type `[4]int` est totalement incompatible avec le type `[5]int`. Plus important encore : **affecter un tableau à une variable ou le passer en paramètre d'une fonction copie l'intégralité de ses éléments en mémoire !**

```Go
package main

import "fmt"

func modifierTableau(t [3]int) {
	t[0] = 999 // Modifie uniquement la COPIE locale sur la pile !
}

func main() {
	// Déclaration avec Zero Values ([0, 0, 0])
	var a [3]int

	// Déclaration avec initialisation et taille déduite par le compilateur (...)
	b := [...]int{10, 20, 30}

	modifierTableau(b)
	fmt.Println(b[0]) // Affiche toujours 10 ! (a et b sont intacts)

	// Contrairement au C, on peut comparer deux tableaux de même type avec == :
	fmt.Println(a == b) // false (compare élément par élément)
}
```

Quand utilise-t-on un `Array` en Go ? Très rarement directement dans les signatures de fonctions, sauf pour des données de taille fixe garantie (ex: un hash cryptographique

SHA-256 de type `[32]byte` ou une adresse IPv4 `[4]byte`), ou en passant explicitement un pointeur vers le tableau (`*[1024]byte`). Dans 99 % des cas en Go, on utilise des **`Slices`**.

---

2. Les `Slices` (`[]T`) : Anatomie mémoire et pièges en production

Une `Slice` n'est pas un tableau : c'est une **vue dynamique et légère** qui pointe vers un tableau sous-jacent (*backing array*) géré en coulisses par Go.

2.1 Sous le capot : Le descripteur de 24 octets (`SliceHeader`)

Sur une architecture 64 bits, toute variable de type `[]T` est représentée en mémoire par une petite structure de **24 octets** composée de 3 champs de 8 octets :

1. **`Data` (`uintptr` - 8 octets) :** Le pointeur mémoire vers le premier élément accessible dans le tableau sous-jacent.
2. **`Len` (`int` - 8 octets) :** La longueur actuelle (nombre d'éléments lisibles via `s[i]`), obtenue avec `len(s)`.
3. **`Cap` (`int` - 8 octets) :** La capacité maximale disponible dans le tableau sous-jacent à partir du pointeur `Data` avant de devoir réallouer, obtenue avec `cap(s)`.

Conséquence : Passer une slice de 10 millions d'entiers à une fonction est instantané car Go ne copie que ces **24 octets** sur la pile !

2.2 Création : `nil` slice vs Empty slice vs `make()`

```go
// 1. Nil slice (Zero Value) : ptr = nil, len = 0, cap = 0
// Aucune allocation mémoire ! Parfaitement valide avec len(), range et append().
var s1 []int // s1 == nil est TRUE

// 2. Slice littérale vide : ptr != nil (pointe vers une adresse spéciale 0 octet), len = 0, cap = 0
s2 := []int{} // s2 == nil est FALSE (utile uniquement si un sérialiseur JSON doit produire "[]" au lieu de "null")

// 3. Pré-allocation avec make([]T, len, cap) — Recommandé en production !
// Alloue un tableau sous-jacent de 100 entiers, mais la slice a une longueur initiale de 0 :
s3 := make([]int, 0, 100) // len(s3) = 0, cap(s3) = 100
```

2.3 Fonctionnement mécanique de `append()`

La fonction native `append(s, elements...)` ajoute des éléments à la fin de la slice selon deux scénarios :

- **Cas 1 : `len < cap` (Espace disponible dans le tableau sous-jacent)** `append` écrit les nouveaux éléments directement dans le tableau existant, incrémente `len` et retourne le descripteur mis à jour. **Zéro allocation mémoire, vitesse maximale.**
- **Cas 2 : `len == cap` (Tableau sous-jacent saturé)** `append` alloue un **nouveau tableau** plus grand sur le tas (en doublant la capacité $\times\ 2$ pour les petites slices, puis avec un facteur de croissance d'environ $\times\ 1.25$ à $\times\ 1.5$ au-delà de 256 éléments), copie les anciens éléments dans ce nouveau tableau, ajoute le nouvel élément, et retourne un descripteur pointant vers la **nouvelle adresse mémoire**.

```
Go
s := make([]int, 0, 2)
s = append(s, 1, 2) // len=2, cap=2 (Même tableau)
s = append(s, 3)    // Dépassement ! Nouveau tableau alloué : len=3, cap=4
(Nouvelle adresse Data)
```

2.4 Les 2 grands pièges du découpage (`Slicing`)

Piège n°1 : Corruption mutuelle par partage du tableau sous-jacent

L'opérateur de découpage `sousSlice := s[debut:fin]` (inclusif sur `debut`, exclusif sur `fin`) **ne copie jamais les données**. Il crée simplement un nouveau descripteur de 24 octets qui pointe vers le **même tableau sous-jacent** !

```
Go
original := []int{10, 20, 30, 40, 50}
vue := original[1:3] // Contient [20, 30], mais sa capacité 'cap(vue)' vaut 4
(jusqu'à la fin d'original) !

// DANGER 1 : Modifier un élément de 'vue' modifie 'original' :
vue[0] = 999 // original vaut maintenant [10, 999, 30, 40, 50] !

// DANGER 2 : Faire un append() sur 'vue' écrase la suite de 'original' car
len(vue)=2 < cap(vue)=4 !
vue = append(vue, 777) // original[3] (qui valait 40) est écrasé par 777 !
```

Comment s'en protéger ?

1. **L'expression de slice complète à 3 indices (`s[debut:fin:max]`) :** Permet de brider la capacité de la sous-slice à sa longueur (`cap = max - debut`). Ainsi, le moindre

`append` sur `vue` forcera l'allocation d'un nouveau tableau indépendant sans jamais écraser `original` :

```go
vueSecurisee := original[1:3:3] // len = 2 (3-1), cap = 2 (3-1) !
```

2. **Clonage explicite avec `copy(dest, src)` ou `slices.Clone(s)` (Go 1.21+) :**

```go
copieIndependante := make([]int, len(vue))
copy(copieIndependante, vue) // Copie physique des octets vers le nouveau tableau
```

Piège n°2 : Fuite mémoire (*Memory Leak*) sur les grosses slices

Si vous lisez un fichier de 100 Mo dans une slice `buffer`, puis que vous retournez uniquement les 10 premiers octets `return buffer[:10]`, le Garbage Collector **ne pourra jamais libérer les 100 Mo** car votre petite sous-slice garde une référence active vers le tableau sous-jacent complet ! Il faut impérativement copier ces 10 octets dans une nouvelle slice dédiée avant de retourner.

---

3. Les `Maps` (`map[K]V`) : Tables de hachage natives

Une `map` est une table de hachage associative non ordonnée intégrée au langage, offrant des opérations de lecture, d'écriture et de suppression en temps constant moyen $O(1)$.

3.1 Règles sur les types de clés (`K`)

La clé `K` peut être de n'importe quel type **comparable** avec l'opérateur `==` : `string`, `int`, `bool`, pointeurs, ou même une `struct` / un `array` (ex: `[16]byte`) tant qu'ils ne contiennent que des champs comparables. En revanche, les types `slice`, `map` et `func` ne sont pas comparables et **ne peuvent pas servir de clé**.

3.2 Initialisation, lecture, écriture et le piège de la `nil map`

Comme une slice, une variable `map` est en réalité un pointeur vers une structure interne du runtime (`hmap`).

```go
// DANGER : Map non initialisée (Zero Value = nil)
var mNil map[string]int
fmt.Println(mNil["inconnu"]) // Lecture sur map nil : AUTORISÉ (renvoie la Zero Value 0)
```

```
// mNil["alice"] = 10        // Écriture sur map nil : CRASH IMMÉDIAT (panic:
assignment to entry in nil map) !

// Initialisation correcte avec make (possibilité de donner un indice de capacité
initiale) :
scores := make(map[string]int, 50)

// Ou avec un littéral :
configs := map[string]string{
      "host": "localhost",
      "port": "8080",
}

// Écriture et suppression
scores["alice"] = 42
delete(scores, "alice") // Sans effet (ne plante pas) si la clé n'existe pas
```

3.3 L'idiome du "Comma Ok" (`val, ok := m[cle]`)

Puisqu'une lecture sur une clé inexistante renvoie silencieusement la *Zero Value* du type valeur (`0` pour `int`, `""` pour `string`, `false` pour `bool`), comment distinguer un joueur qui a un score de `0` d'un joueur qui n'est pas dans la map ?

On utilise la seconde valeur de retour booléenne fournie par l'accès à la map :

```
Go
if score, existe := scores["bob"]; existe {
      fmt.Printf("Bob est inscrit avec %d points\n", score)
} else {
      fmt.Println("Bob n'existe pas dans la map")
}
```

3.4 Caractéristiques internes critiques à connaître en entreprise

1. **Ordre d'itération aléatoire volontaire :** Lorsque vous parcourez une map avec `for cle, val := range maMap`, le runtime Go **randomise volontairement l'ordre de départ** à chaque exécution. Les créateurs de Go ont codé ce comportement pour empêcher les développeurs d'écrire des algorithmes qui dépendent accidentellement de l'ordre d'insertion des clés. Pour afficher une map triée, il faut extraire ses clés dans une slice, trier la slice (`slices.Sort`), puis parcourir la slice.
2. **Interdiction de prendre l'adresse d'un élément (`&maMap["cle"]` est interdit) :** Au fur et à mesure qu'une map grandit, le runtime réalloue ses compartiments mémoire (*buckets*) et déplace les valeurs. Si Go vous autorisait à garder un pointeur `*int` vers une valeur de la map, ce pointeur deviendrait invalide lors du prochain `insert`.

3. **Pattern du "Set" (Ensemble de valeurs uniques) :** Go n'a pas de type `Set` natif. On l'implémente avec une `map[string]bool` ou, pour consommer **0 octet** pour la valeur, avec une structure vide `map[string]struct{}`.